

Response to Reviewers

Manuscript: SHM-EM: A forecast-aware event management framework for heterogeneous engineering monitoring
Manuscript number: SOFTX-D-26-00931

We thank the Editor and Reviewers for the constructive comments. We retained the software architecture and research-software positioning while substantially strengthening validation, specification, related-software positioning, portability evidence, and limitations. The revised manuscript does not introduce a new forecasting algorithm or claim forecasting superiority. The revised software release retains a fixed production-core baseline.

The principal additions are: (i) a real versioned contract example and verified six-model configuration; (ii) a code-derived Project Future State algorithm and controlled-transition sequence; (iii) a synthetic second configuration used solely as a software-reuse fixture; (iv) a 15-case validation matrix comprising one positive control, 12 failure-path cases, and two input-availability controls; (v) repeated runtime and bounded-scaling evidence; (vi) a concrete formal-event provenance trace; (vii) an exercised Docker/Linux logical reproduction path with its numerical non-identity reported explicitly; and (viii) a source-grounded related-software comparison.

Reviewer 1

R1-0. General recommendation

Reviewer comment

I recommend retaining the current software architecture and positioning; I would not recommend adding another forecasting algorithm comparison simply to strengthen the paper. The highest-value revision would be stronger software validation: failure-path testing, quantitative runtime/scalability evaluation, an explicit contract example, a provenance trace demonstration, and ideally a second heterogeneous configuration demonstrating genuine reuse.

Response

We agree and followed this recommendation. The software architecture and three-contribution positioning were retained. We added the requested contract, failure-path, runtime, provenance, and second-configuration evidence rather than adding a forecasting-algorithm comparison.

Changes in manuscript

The Abstract and Section 1 now state the three contributions once. Sections 2.1.2-2.1.4 formalize them. Section 3 adds five validation subsections covering the public case, cross-configuration reuse, failure paths, runtime, and provenance/reproducibility. **Final location: p. 1, lines 9-22; pp. 3-10, lines 57-188.**

Evidence

docs/revision/DATA_MODEL_CONTRACT_SPEC.md;
docs/revision/PROJECT_FUTURE_STATE_ALGORITHM.md; artifacts/revision/benchmarks/route-
p/failure-regression/failure-matrix-v2.md; artifacts/revision/manuscript/final-
performance-table.md; docs/revision/PROVENANCE_TRACE_EXAMPLE.md.

Scope / non-claim

We do not claim a new forecasting algorithm, improved forecasting accuracy, or a global test total obtained by summing overlapping evidence families.

R1-1. Reuse/generalization beyond one excavation

Reviewer comment

The empirical validation is too narrow for several of the paper's reuse/generalization claims. The entire demonstrated workflow is based on one unidentified excavation-monitoring case, and the authors themselves acknowledge that this does not validate other structures or domains. Yet Section 4.2 argues for reuse through minimal registration. At minimum, add a second substantially different monitoring configuration or a synthetic cross-domain example demonstrating that the same backend/event workflow can actually be reused without code modification.

Response

We added one synthetic bridge-monitoring configuration as a software-reuse fixture. It registers one project, three stations, 12 instruments, 26 metric bindings, four observation mappings, 164 feature mappings, two compatible model bundles, and one rule. Seven end-to-end functional checks passed and 1,120 forecast rows were produced. Relative to

the fixed revised production core, this configuration required zero backend business-source changes, zero front-end workflow-source changes, zero PIT_PRE core changes, and zero alterations to existing observation-table schemas.

Changes in manuscript

New Section 3.2 and Table 4 report the registration inventory, outputs, functional checks, and exact zero-change boundaries. Section 4.2 replaces the submitted universal reuse statement with an experiment-specific result. **Final location: pp. 8-9, lines 155-162; p. 11, lines 195-198.**

Evidence

artifacts/revision/benchmarks/route-p/phaselb-regression.json;
artifacts/revision/benchmarks/route-p/phaselb-regression/; Git diff relative to the fixed revised production-core baseline.

Scope / non-claim

The two compatible model bundles were used solely as software-workflow fixtures. Their outputs are not interpreted as bridge-domain predictive validation, cross-domain forecasting accuracy, external field validation, or universal no-code onboarding.

R1-2. Software effectiveness and quantitative criteria

Reviewer comment

The paper demonstrates reproducibility more strongly than software effectiveness. The reported evidence, six successful model runs, 4,960 synchronized records, successful gate checks, hashes, and workflow reproduction, mainly establishes that the pipeline executes correctly. It does not quantitatively demonstrate that SHM-EM improves integration effort, reliability, traceability, latency, or event-management performance relative to a conventional implementation. Add measurable software-level evaluation criteria such as integration/configuration effort, execution latency, event-processing throughput, storage overhead, failure detection, or provenance overhead.

Response

We added measurable software-level evidence for configuration effort, reference workflow latency, Gate inspection, Project Future State, Evaluate, Execute, provenance retrieval, prediction persistence, integrity verification, and failure blocking. The second-configuration inventory supplies a concrete configuration-change measure. The failure matrix and provenance trace quantify blocking and traceability outcomes.

Changes in manuscript

Sections 3.2-3.5 and Tables 4-7 report the new evidence. Section 4 now distinguishes demonstrated integration/audit behavior from unmeasured production effectiveness. **Final location: pp. 7-10, lines 139-188.**

Evidence

artifacts/revision/manuscript/final-performance-table.md;
artifacts/revision/manuscript/software-test-summary.md;
artifacts/revision/benchmarks/route-p/phaselb-regression.json;
artifacts/revision/manuscript/provenance-trace-final.json.

Scope / non-claim

The measurements characterize the reference implementation under documented single-process workloads. We do not claim speedup over an unspecified conventional platform, multi-user production throughput, reliability improvement, or accuracy superiority.

R1-3. Evaluate/Execute failure-path safety

Reviewer comment

The event-management safety mechanism needs stronger validation. The separation between Evaluate and Execute is one of the manuscript's main contributions, but the evidence is essentially a successful reproduction example. Add negative/failure-path experiments: incomplete forecast steps, stale predictions, incorrect model hash, missing required target, invalid units, schema mismatch, temporal misalignment, failed model run, and corrupted prediction batch. Demonstrating that the execution gate correctly blocks each case would substantially strengthen the paper.

Response

We executed a 15-case validation matrix comprising one positive control (P00), 12 failure-path cases (F01-F12), and two input-availability controls (I01-I02) in isolated databases. It includes every requested fault. F09 exposed a persisted-integrity gap: stored hashes had to be recomputed from canonical rows rather than trusted as metadata. A narrowly scoped correction added that revalidation. F12 then confirmed that mutation after Evaluate is detected by Execute's independent reload and Gate recheck. Evaluate retains one evaluation/audit run but has no formal business side effects: it creates no formal event, execution Gate, response workflow or step, notification, report, evidence, or prediction link. All cases expected to be blocked produced zero formal event, response-workflow, response-step, report, evidence, or prediction-link records.

Changes in manuscript

Section 3.3 and Table 5 describe the matrix, failure boundaries, F09 safeguard, and Evaluate-to-Execute mutation case. Fig. 3 separates validation, Evaluate, Execute recheck, formal side effects, and provenance. **Final location: p. 6, lines 107-117; p. 9, lines 163-170.**

Evidence

artifacts/revision/benchmarks/route-p/failure-regression/failure-matrix-v2.md;
artifacts/revision/benchmarks/route-p/failure-regression/failure-matrix-v2.json;
docs/revision/figures/forecast-event-sequence.mmd.

Scope / non-claim

P00 is a valid positive control, so we do not describe all 15 cases as invalid or blocked. The matrix supports only the tested boundaries and is not a safety certification.

R1-4. Differentiation from Predictive-SHM

Reviewer comment

The novelty relative to Predictive-SHM needs sharper differentiation. The manuscript states that Predictive-SHM already supports multi-sensor ingestion and replaceable forecasting models, while SHM-EM addresses the observation-to-model contract, temporal alignment, and controlled forecast-to-event transition. Since this is the closest related software, provide a concise feature-level comparison showing exactly what SHM-EM adds beyond Predictive-SHM and other relevant monitoring/event-processing systems.

Response

We added a primary-source-grounded comparison among OGC SensorThings, generic CEP, Predictive-SHM, and SHM-EM. Predictive-SHM is explicitly credited for multi-source ingestion, ULDM/unified preprocessing, metadata-driven registration, model adapters, pluggable prediction, standardized timestamped forecasts, visualization, and alerting. SHM-EM is positioned as a downstream event-workflow boundary, not as a replacement.

Changes in manuscript

Section 1 and Table 1 now provide the distinction. The text states that Predictive-SHM's primary source does not explicitly report a common multi-model origin or project-level synchronized future timeline. **Final location: pp. 2-3, lines 25-51.**

Evidence

docs/revision/RELATED_SOFTWARE_COMPARISON.md; artifacts/revision/manuscript/related-software-comparison.csv; artifacts/revision/manuscript/related-software-sources.json.

Scope / non-claim

Third-party cells use *Not reported* rather than inferring absence. No cross-system superiority is claimed.

R1-5. Formal data-model contract

Reviewer comment

The claimed "versioned data-model contract" is central but insufficiently formalized in the manuscript. Section 2.1.2 describes feature mappings, input ordering, schema versions, transformations, targets, and bundle hashes, but the reader is not shown a sufficiently concrete contract specification. Include a compact schema/example containing mandatory fields, versioning rules, validation constraints, feature order, units, transformations, target mapping, and failure behavior.

Response

We exported the authoritative database contract and added a compact manuscript extract with timeline, model dimensions, one ordered feature with raw/engineering units, and an input-schema hash. The repository retains the complete source, role, transformation, conversion, validation, version, and fail-closed fields.

Changes in manuscript

Section 2.1.2, Table 2, and Listing 1 formalize the contract and distinguish the 164-feature common pool, each frozen preprocessor's 114- or 164-column aligned input, database model-feature mapping counts, and output targets. **Final location: pp. 4-5, lines 76-96.**

Evidence

docs/revision/DATA_MODEL_CONTRACT_SPEC.md; docs/revision/examples/data-model-contract.example.json; docs/revision/examples/data-model-contract.schema.json; artifacts/revision/manuscript/data-model-contract-export.json.

Scope / non-claim

The compact listing is illustrative; the repository export is authoritative. We do not claim that the current contract supports arbitrary model families without adapter work.

R1-6. Project Future State definition

Reviewer comment

The project future-state concept requires a more precise formal definition. It is described as synchronized multi-target forecasts plus target/station/project risk summaries, but the aggregation from 124 target channels to station-level and project-level states is not sufficiently explicit. Provide either mathematical definitions, pseudocode, or an algorithm showing synchronization, threshold evaluation, aggregation, earliest exceedance determination, and project-state assignment.

Response

We derived Algorithm 1 from the frozen service implementation. It covers policy-hash validation, batch/horizon resolution, engineering-series selection, unit-compatible threshold evaluation, consecutive-step streaks, target/station/timeline aggregation, observed-versus-forecast risk, earliest exceedance, and deterministic state hashing.

Changes in manuscript

Section 2.1.3 now contains Algorithm 1 and precise boundary semantics. Section 3 reports six boundary tests and runtime. **Final location: p. 5, lines 97-106.**

Evidence

docs/revision/PROJECT_FUTURE_STATE_ALGORITHM.md; docs/revision/PROJECT_FUTURE_STATE_SPEC.md; artifacts/revision/manuscript/future-state-boundary-tests.json.

Scope / non-claim

Project Future State is deterministic rule-bound aggregation, not probabilistic calibration, causal inference, or an Execute prerequisite.

R1-7. Six-model reproducibility

Reviewer comment

The six forecasting models are under-described for reproducibility. All are stated to use a Transformer-CNN architecture, but Table 1 mainly provides input/output dimensions. Since model bundles are integral to the reproducible workflow, provide essential architecture/configuration information or point explicitly to immutable model-card/configuration files corresponding to release v1.0.0.

Response

We reconciled model-specific history, aligned input and target counts against the frozen preprocessors, inference scripts, model-weight shapes, and Phase 0.6 numerical matrices. We also report Transformer dimensions, attention heads, feed-forward dimensions, CNN channels/kernel, parameter source, and verified hashes. The database mapping counts previously labelled as inputs are now identified separately.

Changes in manuscript

Table 2 reports the verified aligned-input/output configuration: YD 114/42, XD 114/42, Strain 114/14, Pressure 114/14, Water 114/2, and Settlement 164/10, with 12-16 declared runner histories. Pressure declares 13 rows but its frozen script consumes the final 12 rows ($m=10$, $lag=2$). **Final location: pp. 4-5, lines 76-90.**

Evidence

artifacts/revision/manuscript/MODEL_DIMENSION_RECONCILIATION.md; artifacts/revision/manuscript/model-dimension-reconciliation.json;

docs/revision/MODEL_CONFIG_SUMMARY.md; immutable model cards and hashes under
src/pit_pre/models/.

Scope / non-claim

Unrecorded training parameters are not inferred. Model configuration evidence is not presented as a new accuracy benchmark.

R1-8. Point forecasts

Reviewer comment

The use of only point forecasts is a significant limitation for forecast-aware event management.

Response

We agree and now state this limitation in the Abstract-level scope, model section, validation interpretation, Impact, and Conclusions. The Gate verifies software/data eligibility; it does not transform point forecasts into predictive confidence.

Changes in manuscript

Sections 2.1.3, 4.4, and 5 explicitly exclude calibrated intervals, quantiles, confidence, and probabilistic exceedance. **Final location: p. 5, lines 97-106; p. 11, lines 204-218.**

Evidence

docs/revision/MODEL_CONFIG_SUMMARY.md;
artifacts/revision/manuscript/FINAL_LIMITATION_MATRIX.md.

Scope / non-claim

No uncertainty interval, probabilistic risk, or confidence claim is made.

R1-9. Runtime scalability

Reviewer comment

The manuscript should quantify runtime scalability. The reference case contains only 2,464 observations, nine field points, 124 target channels, 40 future steps, and 4,960 forecast records. Report inference time, future-state construction time, Evaluate/Execute latency, database size or query latency, and preferably scaling experiments with increasing points/targets/forecast records.

Response

We added 30-run reference measurements for prediction, input assembly, inference, conversion, persistence estimate, hashing, Gate, Future State, joint series, Evaluate, and provenance, plus 10 Execute runs. We also report two Gate stress endpoints: 4,960 and 49,600 rows.

Changes in manuscript

Section 3.4 and Table 6 report median, p95, workload, and repetition count. The reference Gate median/p95 are 343.129/407.100 ms. **Final location: pp. 9-10, lines 171-177.**

Evidence

artifacts/revision/manuscript/final-performance-table.md;
artifacts/revision/benchmarks/route-p/.

Scope / non-claim

We do not claim linear scaling, $O(N)$, multi-user capacity, or production throughput.

R1-10. MySQL characterization

Reviewer comment

MySQL scalability is acknowledged but not experimentally characterized. The authors correctly state that high-frequency waveforms may require alternative storage adapters. A small stress/scalability experiment would help establish the practical boundary at which the current MySQL implementation becomes unsuitable.

Response

We added single-run MySQL persistence and independent integrity-verification characterization at 4,960 and 49,600 rows, alongside repeated Gate endpoints. We also documented the exact data-access boundary and the responsibilities of a future adapter.

Changes in manuscript

Section 3.4 reports persistence and integrity timings; Section 4.4 states that MySQL is the only validated backend. The 50,000-row Gate cap is explicitly identified as an application safeguard. **Final location: pp. 9-10, lines 171-177; p. 11, lines 204-212.**

Evidence

artifacts/revision/manuscript/final-performance-table.md;
docs/revision/STORAGE_ADAPTER_BOUNDARY.md.

Scope / non-claim

We do not infer a MySQL capacity limit from two endpoints and do not claim performance or compatibility for an unimplemented time-series database.

R1-11. Security boundary

Reviewer comment

Security is currently a notable deployment limitation. The paper states that the public baseline assumes a trusted network and has no application-level authentication. Since formal engineering events and response workflows can be created, access control, authentication, authorization, audit protection, and API security deserve clearer treatment before presenting the software as suitable for operational environments.

Response

We added a release security policy and operational deployment guidance covering TLS, identity, RBAC, separate Execute authorization, least-privilege database roles, secret management, segmentation, protected audit/provenance storage, rate limits, and backup/restore. We also clarify that SHA-256 is not tamper-proof against a privileged attacker.

Changes in manuscript

Section 4.4 now identifies the research-reference security boundary and recommended controls. **Final location: p. 11, lines 204-212.**

Evidence

SECURITY.md.

Scope / non-claim

No built-in application authentication, security certification, secure-by-default claim, or privileged-attacker resistance is asserted.

R1-12. Linux and container portability

Reviewer comment

Reproducibility appears strong but is currently Windows-centric. The reproduction workflow was validated only on Windows 10/11 with PowerShell 7. For open research software using Java, Python, MySQL, Vue, and Node.js, Linux validation and/or containerized deployment would substantially improve portability. A Docker/Docker Compose reproduction environment would be valuable.

Response

We added and exercised Docker/Docker Compose component and logical end-to-end workflows. The Docker/Linux run completed six models, 124 targets, 40 steps, 4,960 rows, Gate, Project Future State, Evaluate, Execute, and provenance. Input and model-contract hashes matched, and 4,960/4,960 rows matched structurally. The normalized output hash differed from Windows (exactPredictionReproduction=false); the maximum persisted absolute difference was 0.00285349; no tolerance was introduced (toleranceApplied=false).

Changes in manuscript

Metadata C6, Sections 3.5 and 4.4, and the availability/reproduction guidance now report this bounded result. **Final location: p. 10, lines 178-188; p. 11, lines 204-212.**

Evidence

compose.yaml; src/backend/Dockerfile; src/frontend/Dockerfile; src/pit_pre/Dockerfile;
artifacts/revision/portability/cross-platform-comparison.json;
artifacts/revision/portability/cross-platform-numeric-difference.json; full row-wise comparison artifact.

Scope / non-claim

Windows remains the exact-output reference. The container result is reported only as functional/logical end-to-end portability; numerical output identity and separately captured native Ubuntu-host validation are not established.

R1-13. Validation/evaluation/execution sequence

Reviewer comment

Figure 3 communicates the core contribution well, but the distinction between validation, rule evaluation, and execution eligibility should be made even clearer. These are different safeguards and currently risk appearing conceptually overlapping. A sequence diagram showing Forecast → Persist → Validate → Evaluate → Recheck Gate → Execute → Provenance would make the software behavior more rigorous.

Response

We replaced the ambiguous mechanism flow with a code-crosschecked sequence. It separates contract/integrity validation, optional Future State inspection, rule semantic validation, Evaluate candidate calculation plus its audit-run insertion, Execute Gate recomputation/persistence, formal business side effects, and provenance.

Changes in manuscript

Section 2.1.4 and revised Fig. 3 define the sequence and state that Future State is an independent read path rather than an Execute precondition. **Final location: p. 6, lines 107-117.**

Evidence

docs/revision/figures/forecast-event-sequence.mmd;
artifacts/revision/manuscript/sequence-code-crosscheck.json.

Scope / non-claim

The sequence does not imply that Evaluate authorizes Execute or that Project Future State is required for formal execution.

R1-14. Figure 4 screenshots

Reviewer comment

Figure 4 demonstrates that a functional user interface exists, but the screenshots contribute relatively little scientific evidence. Consider reducing the space devoted to screenshots and using some of that space for quantitative software evaluation, failure-path testing, or contract examples.

Response

We reduced the three submitted screenshot pages to one compact three-panel composite and reassigned manuscript space to the contract, algorithm, failure matrix, runtime, reuse, and provenance evidence.

Changes in manuscript

Revised Fig. 4 contains Project Workspace, Observation and Prediction, and Prediction Runs in one 175-mm-wide composite with a caption that identifies the UI as illustrative. **Final location: pp. 6-7, lines 124-133.**

Evidence

artifacts/revision/manuscript/FIGURE4_REDUCTION_PLAN.md.

Scope / non-claim

The interface screenshots are not used as scientific validation evidence.

R1-15. Unsupported Impact claims

Reviewer comment

Section 4 (“Impact”) currently contains several assertions that are not directly demonstrated. For example, “reuse does not require changes to the core back end, front end, event workflow, or existing source tables” is a strong maintainability/extensibility claim. Validate this experimentally by integrating a second project/model configuration and reporting exactly which files/configurations had to be modified.

Response

We removed the universal wording and rewrote Impact around measured evidence. The revised statement begins “In the synthetic second-configuration experiment” and reports the exact registration inventory, functional results, and zero frozen-core/schema modifications.

Changes in manuscript

Section 4 is restructured as reproducible integration, tested cross-configuration reuse, controlled transition/traceability, and deployment/scientific scope. **Final location: pp. 8-9, lines 155-162; pp. 10-11, lines 190-212.**

Evidence

artifacts/revision/manuscript/IMPACT_RESTRUCTURING_PLAN.md;
artifacts/revision/benchmarks/route-p/phaselb-regression.json.

Scope / non-claim

No universal no-code, arbitrary-deployment, cross-domain prediction, or reliability-improvement claim remains.

R1-16. Software testing summary

Reviewer comment

The manuscript needs an explicit software testing summary. It states that backend tests, frontend checks/builds, and PIT_PRE contract tests passed, but does not report the number/type of tests or coverage. A small table reporting unit tests, integration tests, API tests, contract-validation tests, negative tests, and end-to-end tests would improve confidence in the software.

Response

We added a family-level testing table: 55 backend tests, 13 PIT_PRE tests, a 15-case validation matrix (P00, F01-F12, I01-I02), seven second-configuration checks, two front-end checks, and one public reproduction.

Changes in manuscript

Table 3 reports each family independently and explains why the families are not summed. **Final location: pp. 7-8, lines 139-143.**

Evidence

artifacts/revision/manuscript/software-test-summary.md;
artifacts/revision/manuscript/software-test-summary.json.

Scope / non-claim

We do not report a code-coverage percentage because no stable coverage instrument is part of the release, and we do not double-count overlapping evidence as one total.

R1-17. Concrete provenance trace

Reviewer comment

The provenance mechanism is promising but deserves a concrete reproducibility demonstration. Select one generated formal event and show how its event ID can be traced back through rule version → prediction batch → model version/hash → input window → forecast values. This would turn provenance from an architectural claim into demonstrable functionality.

Response

We captured formal event FEVT-4-f61b7667dcc01721aa2a and traced it to rule version v2, batch 40, run 236, settlement model version/hash, input window/schema hash, selected 40-step forecast, Gate 1, first exceedance, and response workflow. The isolated database was restored after export.

Changes in manuscript

Section 3.5 and Table 7 present the trace and distinguish API-visible metadata from independently queried persisted-integrity fields. **Final location: p. 10, lines 178-188.**

Evidence

docs/revision/PROVENANCE_TRACE_EXAMPLE.md; artifacts/revision/manuscript/provenance-trace-final.json.

Scope / non-claim

We do not claim that every API directly exposes every persisted hash.

R1-18. OGC SensorThings relationship

Reviewer comment

The relationship with OGC SensorThings and existing standards could be clearer. The paper correctly states that such standards do not define model-specific input contracts or forecast-to-event transitions, but it should explain whether SHM-EM can ingest/interoperate with SensorThings-compliant observations or whether its observation registry represents a separate abstraction.

Response

We now describe SensorThings as a standardized observation/sensor Web model and the SHM-EM registry as a separate internal abstraction. A future adapter could map SensorThings entities and observations into the registry, but no endpoint, adapter, or conformance test exists in v1.0.1.

Changes in manuscript

Section 1 and Section 4.4 state the current non-conformance boundary and prospective adapter role. **Final location:** pp. 2-3, lines 25-51; p. 11, lines 204-212.

Evidence

docs/revision/SENSORTHINGS_POSITIONING.md; docs/revision/RELATED_SOFTWARE_COMPARISON.md.

Scope / non-claim

No SensorThings compatibility, ingestion, API conformance, or Annex A conformance is claimed.

R1-19. Repetition of contributions

Reviewer comment

The manuscript is generally well organized, and Figures 1–3 provide a coherent architectural narrative. However, some sections repeatedly describe the same three contributions—data-model contract, project future state, and controlled event transition. Condensing repetitive descriptions would create space for stronger experimental evidence.

Response

We condensed the recurring contribution language into one Introduction statement, one mechanism definition per Section 2 subsection, one evidence treatment in Section 3, and one synthesis in the Conclusion.

Changes in manuscript

The submitted Section 4 prose was replaced with evidence-driven impact and explicit limitations; Fig. 4 was compressed to make room for the new tables and algorithm. **Final location:** pp. 2-3, lines 25-51; p. 11, lines 213-218.

Evidence

artifacts/revision/manuscript/REPETITION_REDUCTION_MAP.md;
artifacts/revision/manuscript/FIGURE4_REDUCTION_PLAN.md.

Scope / non-claim

Persisted-result integrity remains a safeguard under contribution 3 and is not introduced as a fourth contribution.

Reviewer 2

R2-1. Differentiation and empirical benchmark

Reviewer comment

The authors recently published a related paper on SHM in this journal (<https://doi.org/10.1016/j.softx.2026.102732>), but only briefly mention it without clearly differentiating the contributions of the two works. To better position this framework, I recommend explicitly contrasting SHM-EM with their previous submission. Furthermore, there is no empirical benchmark comparing SHM-EM against existing platforms (e.g., Predictive-SHM) in terms of integration effort, computational overhead, or forecasting accuracy.

Response

We agree that empirical software comparison was needed. Because SHM-EM does not introduce a forecasting algorithm, the revision evaluates integration/configuration effort, software-level runtime, failure handling, and provenance rather than attributing predictive accuracy to the event-management framework. We also added a source-grounded responsibility comparison and explicitly credit Predictive-SHM's upstream ingestion, preprocessing, adapter, forecasting, visualization, and alerting functions.

Changes in manuscript

Section 1 and Table 1 differentiate the systems. Sections 3.2-3.5 add measurable SHM-EM software evidence. **Final location:** pp. 2-3, lines 25-51; pp. 7-10, lines 139-188.

Evidence

docs/revision/RELATED_SOFTWARE_COMPARISON.md; artifacts/revision/manuscript/final-performance-table.md; second-configuration, failure, and provenance artifacts.

442 **Scope / non-claim**
 443 We do not compare Predictive-SHM and SHM-EM forecasting accuracy or total runtime because their responsibilities
 444 are not equivalent, and we do not claim cross-system superiority.

445 R2-2. Related-framework comparison table

446 **Reviewer comment**
 447 *I suggest adding a brief comparison table against related frameworks (e.g., Predictive-SHM or generic CEP*
 448 *systems) to show what SHM-EM uniquely solves versus what it shares with existing solutions.*

449 **Response**
 450 We added a compact eight-dimension table covering OGC SensorThings, generic CEP, Predictive-SHM, and SHM-
 451 EM. The repository retains the full comparison and source notes.

452 **Changes in manuscript**
 453 Table 1 appears in Section 1 immediately after the related-software discussion. **Final location: pp. 2-3, lines 25-51.**

454 **Evidence**
 455 artifacts/revision/manuscript/related-software-comparison.md;
 456 artifacts/revision/manuscript/related-software-sources.json.

457 **Scope / non-claim**
 458 No unsupported No value is used for third-party software; *Not reported* is not treated as absence.

459 R2-3. Missing/dropped observations

460 **Reviewer comment**
 461 *Please clarify how missing or dropped sensor data is handled in the rolling forecast windows. This is a*
 462 *common real-world problem in structural monitoring but isn't explicitly addressed before model inputs are*
 463 *assembled.*

464 **Response**
 465 We documented the actual frozen input policy: backward-asof matching within one cadence, followed by the declared
 466 interpolation/boundary-fill policy, with signed source offsets, fill counts/ratios, and gap diagnostics retained in the
 467 input snapshot. A resolvable partial gap may be filled; an entire unavailable required feature or unresolved window is
 468 rejected before inference. Freshness is checked separately before Execute.

469 **Changes in manuscript**
 470 Section 2.1.2 adds a dedicated missing/asynchronous observations subsection; Table 5 includes I01 and I02. **Final**
 471 **location: p. 5, lines 91-96; p. 9, lines 163-170.**

472 **Evidence**
 473 src/pit_pre/pit_pre/features.py; docs/revision/DATA_MODEL_CONTRACT_SPEC.md;
 474 artifacts/revision/benchmarks/route-p/failure-regression/failure-matrix-v2.md.

475 **Scope / non-claim**
 476 We do not claim robustness to arbitrary dropout or apply an unrecorded universal imputation policy.

477 Reviewer 3

478 R3-1. MySQL and data-access abstraction

479 **Reviewer comment**
 480 *The authors use MySQL as the core storage engine. While the paper describes low-frequency data in Section*
 481 *4.4, MySQL may struggle with high-frequency time-series workloads. The authors should briefly clarify the*
 482 *abstraction level of their data access layer and how easy it would be to integrate time-series native databases.*

483 **Response**
 484 We separated the logical observation contract, approved adapters/registry, and MySQL-specific implementation. A
 485 future adapter must preserve logical identifiers, deterministic ordering/time zones, raw and engineering values, units,
 486 quality, conversion provenance, query semantics, PIT_PRE input alignment, and prediction/event transaction and
 487 integrity behavior.

488 **Changes in manuscript**

Sections 2.1 and 2.1.1 identify MySQL as the only validated backend and describe the extension boundary. Section 4.4 lists the unvalidated alternative-TSDB limitation. **Final location: pp. 3-4, lines 58-75; pp. 9-10, lines 171-177; p. 11, lines 204-212.**

Evidence

docs/revision/STORAGE_ADAPTER_BOUNDARY.md.

Scope / non-claim

No TimescaleDB, InfluxDB, or other alternative adapter has been implemented or validated, and the 50,000-row Gate cap is not a MySQL limit.

R3-2. Recommended security patterns

Reviewer comment

Section 4.4 states that the baseline assumes a "trusted network and no application-level authentication." In modern IoT infrastructure software, security is a major concern. The authors should add 2–3 sentences detailing recommended security patterns.

Response

We added explicit recommended patterns: TLS termination, OIDC/OAuth2 or equivalent identity, RBAC with separate Execute privilege, least-privilege storage roles, secret management, network segmentation, protected provenance/audit storage, rate limiting, gateway logging, and tested backup/restore.

Changes in manuscript

Section 4.4 summarizes these controls and retains the research-reference boundary. **Final location: p. 11, lines 204-212.**

Evidence

SECURITY.md.

Scope / non-claim

These are deployment requirements, not controls implemented by v1.0.1.

R3-3. Asynchronous sampling, latency, and missing points

Reviewer comment

The "Project Future State" aligns multiple models to a single prediction origin. In heterogeneous monitoring networks, real-world physical sensors often suffer from network latency or asynchronous sampling. Clarify how it handles missing input points, uneven time steps, or temporal alignment before feeding inputs into the feature vectors.

Response

The common source window is placed on the registered 3-min cadence and shared prediction origin. Features use backward-asof matching within one cadence, followed by declared interpolation/boundary fill. Signed source offsets and fill diagnostics make asynchronous contributions inspectable. Model-specific contracts then select the final 12-16 rows and their ordered feature subsets. Unresolved required inputs fail before inference; stale otherwise-complete batches fail separately at Execute.

Changes in manuscript

Section 2.1.2 now distinguishes the 16-step common window, 12-16 model histories, alignment/fill policy, input completeness, and execution freshness. **Final location: pp. 4-5, lines 76-96.**

Evidence

src/pit_pre/pit_pre/features.py; src/pit_pre/pit_pre/pipeline.py;
docs/revision/DATA_MODEL_CONTRACT_SPEC.md.

Scope / non-claim

The policy is deterministic and versioned; it is not a claim of statistical optimality for arbitrary asynchronous networks.

R3-4. Linux/Docker portability

Reviewer comment

Reproduction is currently validated only on Windows 10/11 using PowerShell 7. For broader open-source adoption in CS communities, Docker containerization is the standard. Providing a docker-compose.yml

environment or at least discussing Linux/Bash containerization steps would significantly increase the software's impact.

Response

We added Docker/Docker Compose definitions and exercised Linux-container component and logical end-to-end paths. The logical workflow completed through six-model inference, 4,960 persisted rows, Gate, Project Future State, Evaluate, Execute, and provenance. Input and model-contract hashes matched. The normalized output hash did not match Windows (exactPredictionReproduction=false), no tolerance was applied (toleranceApplied=false), and the complete row-wise comparison is retained.

Changes in manuscript

Metadata C6 and Sections 3.5 and 4.4 report the positive portability evidence and the numerical limitation together. **Final location: p. 10, lines 178-188; p. 11, lines 204-212.**

Evidence

compose.yaml; the three component Dockerfiles;
artifacts/revision/portability/PHASE2C_COMPLETION_REPORT.md;
artifacts/revision/portability/cross-platform-comparison.json;
artifacts/revision/portability/cross-platform-numeric-difference.json.

Scope / non-claim

Windows remains the exact-output reference. The container result establishes functional/logical workflow portability, while numerical output identity and a separately captured native Ubuntu-host result remain outside the demonstrated evidence.